

▷ Clever Fib(n, m) takes only $20 \lg m \cdot \lg n \times 2$
= $40 \lg m \cdot \lg n$ steps & $\lg m \cdot \lg n$ space.
logarithmic $\ll$ linear $\ll$ exponential.
($\lg n$) (n) (2^n)

Time Complexity of an Algorithm

— Defn: It's the # instructions required, in the worst-case, as a function of the input-size (say, n).

— Detour into multiplication $a * b \bmod m$:
 takes $\approx (\lg m)^2$ time.
 → Advanced algo takes $\approx \lg m$ time.] $\approx \overset{\uparrow}{\lg m} \overset{\uparrow}{\lg m}$ bits

$\lg$ s. Algos	Input-size	Complexity
1) ItFib(n, m)	$(\lg m + \underline{\lg n})$ bits	$\approx \underline{n \lg m}$ [exp.]
2) CleverFib(n, m)	"	$\approx 40. \lg m \cdot \lg n$ [quad]
3) Sort $A[0 \dots n]$	n words	$\approx \frac{n^2}{2}$; $\approx n \cdot \lg n$
4) Test " "	"	$\approx n$ [almost-linear]

Issue in comparisons: $40\lg n > 3n$
for small n !

But, $\triangleright 40\lg n < 3n$, for all $n \geq n_0 := 128$.

$\Rightarrow$ Don't look at small n 's, but growing n .

$\Rightarrow$ You should compare the asymptotics of the functions in n ; $n \rightarrow \infty$.

- Ex. Algo. A runs in time $t_A(n) = 5n^2 + n + 10$.

" B " " " $t_B = n^2 - 1$.

" C " " " $t_C = 100n^{1.5} + 1000$.

Qn: Which among B, C is a better improvement over A?

$$\triangleright \lim_{n \rightarrow \infty} t_B/t_A \rightarrow 1/5 ; \lim_{n \rightarrow \infty} t_C/t_A \rightarrow 0.$$

$\Rightarrow \triangleright C$ is far better than A, B .

[Caveat: Beware of astronomical constants in practice.]

— While comparing:

Obs. 1: Ignore additive/multiplicative constants.

Obs. 2: Identify the leading monomial in n .

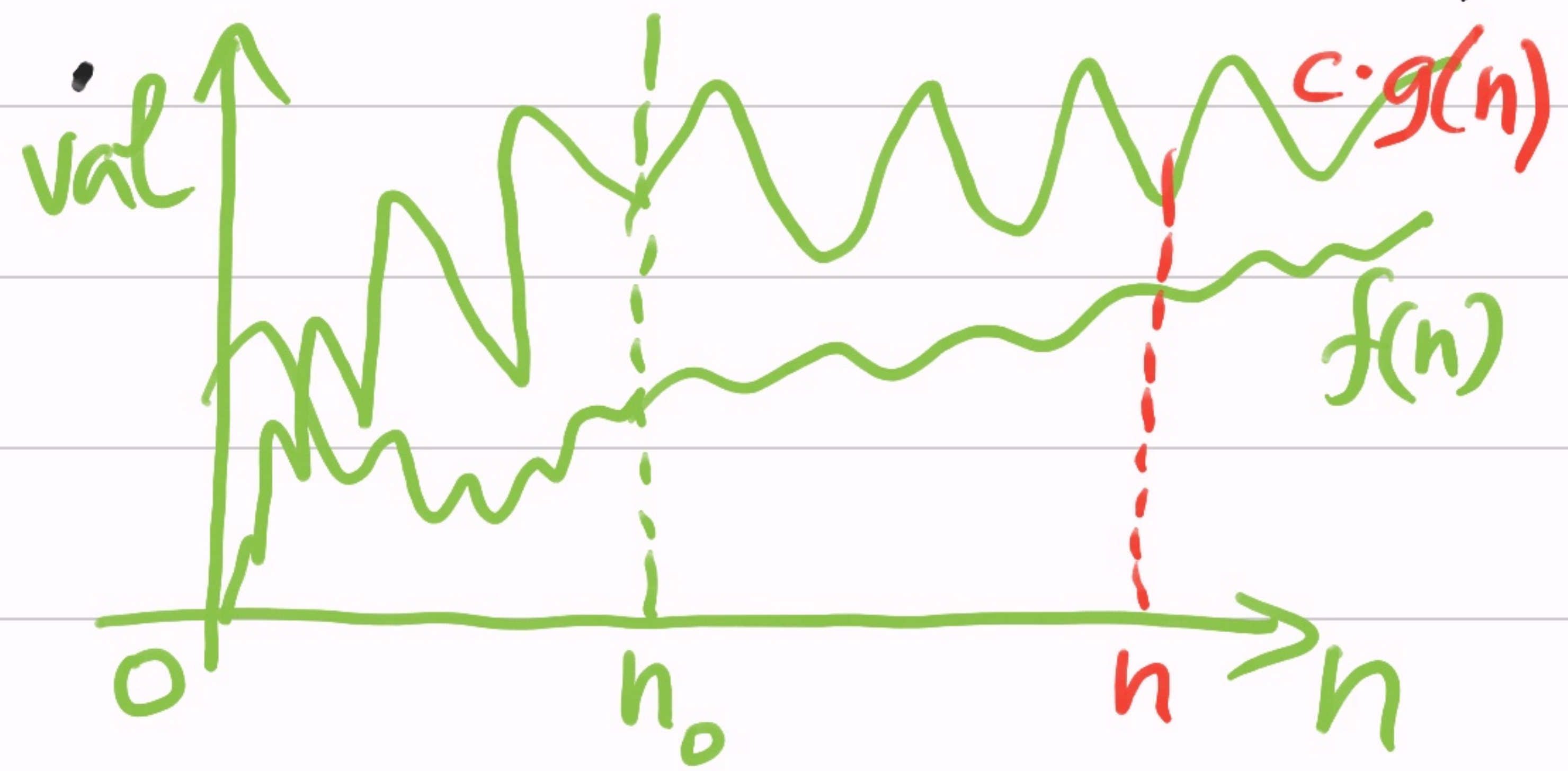
— Let's use a shorthand to do this:

Order Notation

Defn: Fns $f, g: \mathbb{N} \rightarrow \mathbb{R}_{>0}$
 $n \mapsto f(n), g(n)$

$f(n)$ is of the order of $g(n)$ if $\exists c, n_0 \in \mathbb{R}_{>0}$,
 $\forall n \geq n_0, f(n) \leq c \cdot g(n)$

Write $f = O(g)$ [Big Oh].



- eg. $5n^2 + n + 10 = O(n^2)$
 $100n^{1.5} + 1000 = O(n^{1.5}) \leftarrow \text{tight / optimal}$
 $= O(n^2) \leftarrow \text{loose}$

$100 = O(1)$. [$O(1) \neq 100$ is undefined.]

- In this course, we'll prefer^a $g(n)$ that is tight.

Proposition: (i) $f = O(g)$, $g = O(h) \Rightarrow f = O(h)$.

(ii) $f, g = O(h) \Rightarrow \begin{cases} f+g = O(h), \\ f \cdot g = O(h^2). \end{cases}$

(iii) $f = O(h)$, $g = O(1) \Rightarrow f \cdot g = O(h)$.

Pf: [Exercise].

- Ex. $f = 3^n$, $g = 2^n \Rightarrow g = O(f)$; $f \neq O(g)$.
[$f/g \rightarrow \infty$]

— If, $t_C = O(t_B)$ & $t_B \neq O(t_C)$

$\Rightarrow t_C$ is smaller order $\Rightarrow C$ is better.

Asymptotics:

▷ Insertion-sort : $O(n^2)$ time.

Merge-sort : $O(n \lg n)$ time.

Binary-search : $O(\lg n)$ time

Clever Fib(n, m): $O(\lg n \cdot \lg m)$ time.

→ Designing (asymptotically) fast algos. is
an Art!

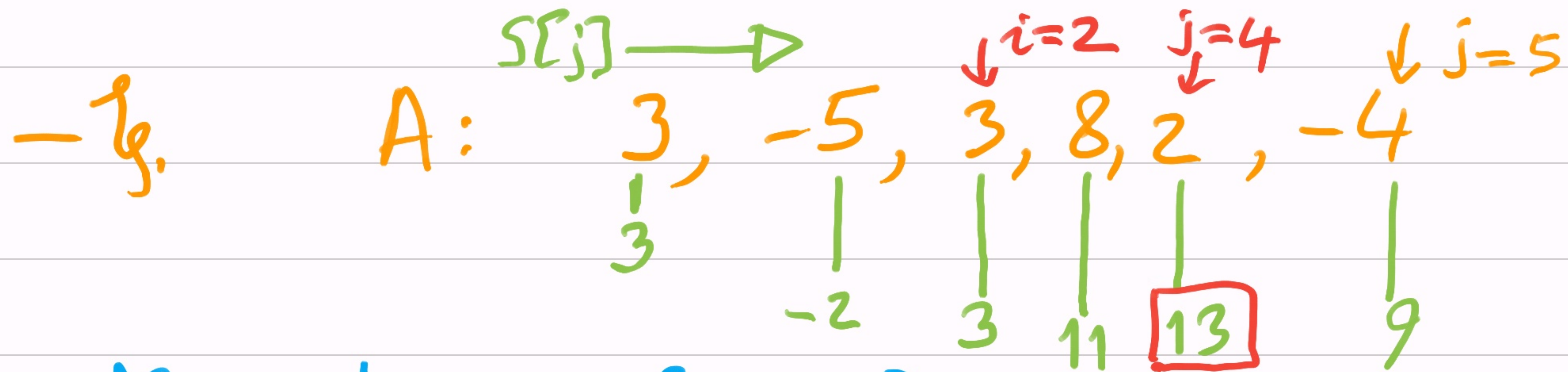
$$\triangleright \# \text{steps} \approx \sum_{i,j=0}^{n-1} (j-i+1) \leq O(n^3).$$

Qn: Can you reduce it to linear-time $O(n)$?

- What if we reduce the number of indices from 2 to 1? $(i, j) \mapsto j$?

$\hookrightarrow$ Reduce the search-space.

Defn: $S[j]$:= Sum of the max-sum subarray ending at j .



▷ If we have $S[0 \dots n-1]$ then we can output max-sum subarray of A in $O(n)$ time.

Pf: Output $\max(S[0 \dots n-1]) = \text{max-sum subarray of A}$. ($\because$ each j appears in S .) $\square$

- Qn: Compute $S[j]$ in $O(1)$ time; given the earlier values?
 $\Rightarrow$ gives an $O(n)$ time algo.!

▷ If $B = A[i \dots j]$ achieves the optimum $S[j]$,
then there are two options:

$B' \cup \{A[j]\}$, where

▷ $B' := A[i \dots j-1]$ achieves
the sum $S[j-1]$

$\{A[j]\}$

X

Pf: • To get $S[j]$ either you use a prefix
before $A[j]$, or you don't.
• The latter if done $\Leftrightarrow S[j-1] < 0$.

□

Lemma (Update): $S[j-1] > 0 \Rightarrow S[j] = S[j-1] + A[j]$,
and $S[j-1] \leq 0 \Rightarrow S[j] = A[j]$.

Pf:
• In case-1: $A[i \dots j-1]$ extends to $A[i \dots j]$.
• In case-2: " is modified completely to $A[j]$. $\square$

Exercise: Use this to calculate the $I[]$ array
s.t. $I[j] = i$ with $S[j] = \text{sum of } A[i \dots j]$.

Max-Sum-Subarray ($A[0 \dots n-1]$): $S[0] \leftarrow A[0];$

$O(n)$ $\{$ • for $j = 1 \dots n-1$ $\{$
if $(S[j-1] > 0)$ $S[j] \leftarrow S[j-1] + A[j];$

```
        } else S[j] ← A[j];  
    }  
O(n) { • Find max S[j] // say, S[j₀].  
        • Output j (j₀, S[j₀]);
```

▷ time $\approx O(n)$.

— Exercise: Adapt the pseudocode to calculate the $I[0 \dots n-1]$ array (of the correct i 's.)